

Flappy Bird

一、写在前面

经过课堂上的学习和课后的阅读，相信大家对强化学习及其相关的 Q-Learning 算法已经有了一定的了解。为了巩固这一阶段学习成果，在本次的 Final Project 中，我们将尝试为 Flappy Bird 这个游戏设计一个智能体，通过强化学习的方式使其能够在这个游戏中获得尽可能高的分数。

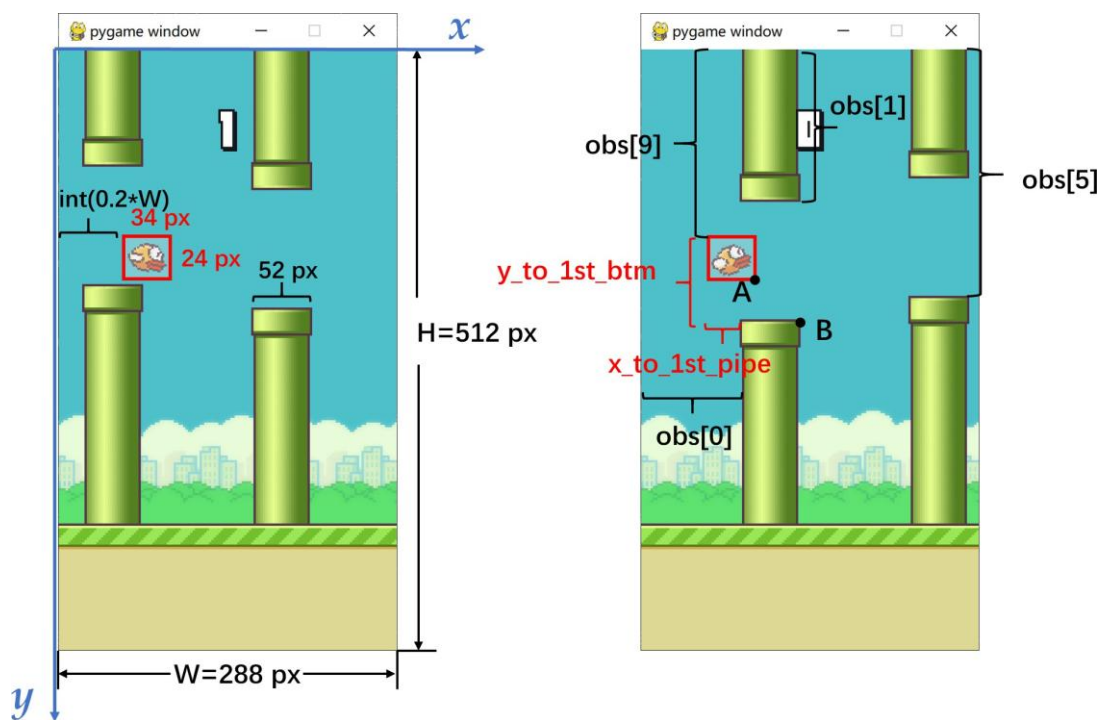
二、Flappy Bird 游戏介绍

Flappy Bird 是一款 2013 年发布的经典 2D 游戏。游戏简单而富有挑战性，玩家需要控制小鸟穿过上下起伏的障碍物（管道），避免碰撞并尽可能飞得更远。游戏的玩法简单介绍如下。

游戏开始时，小鸟从左侧向右飞行，同时会不断地向下坠落。玩家通过点击按键来使小鸟向上飞行一定高度（表现为小鸟的 Flap 动作）。每次点击小鸟都会向上飞行一定高度，并在之后由于重力作用逐渐下落。在飞行的过程中，小鸟会遇到从上方和下方伸出的障碍物（管道）。玩家需要控制小鸟穿过管道之间的空隙，避免碰撞到管道，否则游戏结束。每当小鸟穿过一对管道之间的空隙时，玩家会累积一分。

这里我们通过 `pip install flappy-bird-gymnasium` 引入游戏模块，关于游戏设计的更多文档可以参考 <https://github.com/markub3327/flappy-bird-gymnasium/tree/main>。

三、关于 `process_obs(obs)` 函数



如上图所示，游戏界面的大小为 512 x 288，管道的宽度为 52，小鸟的大小为 24 x 34，同时小鸟距离左边缘的距离为 $\text{int}(0.2*W)$ 。

- `obs[0]`: the last pipe's horizontal position
- `obs[1]`: the last top pipe's vertical position
- `obs[2]`: the last bottom pipe's vertical position
- `obs[3]`: the next pipe's horizontal position
- `obs[4]`: the next top pipe's vertical position
- `obs[5]`: the next bottom pipe's vertical position
- `obs[6]`: the next next pipe's horizontal position
- `obs[7]`: the next next top pipe's vertical position
- `obs[8]`: the next next bottom pipe's vertical position
- `obs[9]`: player's vertical position
- `obs[10]`: player's vertical velocity
- `obs[11]`: player's rotation

游戏环境返回的 12 个观测值如上所述，**the last pipe** 表示画面可见的第一个管道，**the next pipe** 表示接下来的下一个管道，**the next next pipe** 表示接下来的下下一个管道。

如何表示在当前时刻的游戏状态？

一种可行的示例方法是，将 `state` 表示为 $(x_to_lst_pipe, y_to_lst_btm, player_v)$ ，其中 `x_to_lst_pipe` 表示小鸟最左端和最近的第一个管道最左端之间的距离（不考虑已经通过的管道），`y_to_lst_btm` 表示小鸟顶端和最近的第一个底部管道之间的高度差，`player_v` 表示小鸟的垂直速度。因为 `obs[0-11]` 都是归一化后的值，处于 $[0, 1]$ 区间，所以这里会将 `x_to_lst_pipe`, `player_y`, `player_v` 等状态值以 `obs_mul_factor=30` 的倍数放大取整，因为 `state` 需要表示为整数元组才能作为 Q-Function 的 key。（`obs_mul_factor` 的取值同学们也可以当做一个参数自行修改）。

如何定义离小鸟最近的第一个管道？

比较图中 A 和 B 两点的 x 坐标，如果 $x_A > x_B$ ，说明小鸟已经基本通过了 the last pipe，所以离小鸟最近的第一个管道应该定义为 the next pipe；如果 $x_A \leq x_B$ ，此时小鸟还没有完全通过 the last pipe，所以离小鸟最近的第一个管道定义为 the last pipe。

四、关于 reward

- +0.1 - every frame it stays alive
- +1.0 - successfully passing a pipe
- -1.0 - dying
- -0.5 - touch the top of the screen

```
next_obs, reward, terminated, _, info = env.step(action)
    if reward == -1:
        reward = -1000
```

游戏环境返回的 reward 如上所示，在目前的代码中，我们将小鸟死亡导致的-1 的 reward 扩大为-1000，相当于加大了惩罚，目的是希望它尽可能地活下去以拿到更多的分数。

五、Q-Learning

Q-Learning 是一种基于样本的 Q-Value 迭代方法，它是一种无模型的强化学习方法，我们只需要从真实世界中采样足够多的四元组样本（当前状态，行动，新状态，回报），便能较好地估计每一个（状态，行动）对的 Q-Value。接下来我们使用 s 表示当前状态， a 表示作出的行动， s' 表示在状态 s 执行行动 a 后转移到的新状态， r 表示在这一过程中获得的回报，Q-Learning 算法的具体步骤如下：

- 1) 对于所有的 (s, a) 对，初始化 Q-Value 为 0
- 2) 从真实世界采样，获得一个样本 (s, a, s', r)

- 3) 更新 Q-Value:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')] \quad (1)$$

- 4) 回到步骤 (2) 直到 Q-Value 收敛

在公式 (1) 中， α 是学习率（Learning rate），它表示利用新样本对 Q-Value 进行更新的速度，方括号 $[\cdot]$ 中的内容表示根据新样本对 Q-Value 作出的新的估计，它由当前行动后收到的回报 r 和对未来（从新状态 s' 开始）能够收到的所有回报的估计 $\max_{a'} Q(s', a')$ 两部分组成， γ 是一个折扣因子（Discount factor）。

六、作业内容

1. **代码补充(必做)**：在 `src/q_learning.py` 中，已经写好了 `GameAI()` 类的代码框架，请你根据 `TOD01-5` 的提示和相关的函数说明实现各个部分的代码。
2. **状态表示代码修改(bonus1)**：对于 `process_obs()` 这个极为重要的观测值处理函数，你可以尝试自己设计一个更合理的状态表示，使得 `GameAI()` 在相同的训练参数下能有更好的表现。
3. **训练参数调整(必做)**：可以通过适当调整 `alpha`, `gamma`, `epsilon`, `iteration`, `mul_obs_factor` 等参数的大小，比较一下不同参数设置下的模型表现。
4. **reward 调整(bonus2)**：可以尝试调整游戏环境在不同情况下返回的 reward，比较一下模型表现是否有预期的变化。
5. 选择一个输出平均分数最高的模型，将其重命名为 `q_best.pkl` 进行提交。

七、提交要求

请提交一个以 `final_project_学号.zip` 命名的 zip 格式压缩文件，此压缩文件应当包含：

- 1) 补充完整的 `q_learning.py`;
- 2) 训练得到的最好的 `q_best.pkl`;
- 3) 文件夹 `src`;
- 4) 作业报告 `report.pdf`。

注意，请在你补充的代码中必要的地方写上注释。

作业报告要求：

- 1) 结合 `GameAI()` 的代码框架，说明强化学习和 Q-learning 是如何帮助智能体提高 Flappy Bird 的游戏分数。
- 2) 如果完成了 `bonus1`，请在报告中描述你基于观测值 `obs` 的状态设计方法，简要说明一下设计思路，并且与示例方法进行一下比较。
- 3) 通过调整参数比较模型的性能表现，分析不同参数可能产生的影响。
- 4) 如果完成了 `bonus2`，请在报告中说明你的 `reward` 分配方法，并且与原本的 `reward` 分配方法进行一下比较。
- 5) 报告的具体格式没有要求。

八、成绩评定

代码补充实现占 50%，`report` 占 50%。

九、Q&A

1. **如何进行训练？** 进入 `src` 目录，执行如下命令，

```
python .\train_ai_or_play.py --train
```

2. **如何验证模型的性能表现？** 进入 `src` 目录，在 `train_ai_or_play.py` 中修改 `path`，然后执行如下命令，

```
python .\train_ai_or_play.py --no-train
```

3. 使用 `src` 的默认参数和**示例方法**，在 AMD Ryzen 7 5800H 上的训练时间大约为 7 min and 27 sec。模型的平均分数为 32.2。

```
Playing training game 50000
Done training
Training time: 7 min and 27 sec
ryan@machine > D:\Projects\Python\Homework\FlappyBird-src > py311 3.11.5
This try gets 62 score(s).
This try gets 24 score(s).
This try gets 5 score(s).
This try gets 2 score(s).
This try gets 68 score(s).
The average score(s) of Q-Function: 32.2
```

4. 其他可能出现的问题或者未能考虑到的事宜，请及时联系助教。